

Rapport Détaillé de Présentation

Robot Hiwonder JetMax

Bras manipulateur articulé intelligent sous NVIDIA Jetson Nano et ROS

Projet de Robotique Appliquée
Arts et Métiers Campus Rabat

Année universitaire 2025–2026

Structure générale : La présentation `presentation_jetmax.pdf` compte **46 pages** réparties en **14 sections**. Elle couvre l'ensemble du projet depuis la mise en route physique du robot jusqu'aux applications de vision par intelligence artificielle, suivant une progression pédagogique : installation matérielle → prise en main → fondements théoriques → applications avancées.

Table des matières

Pages 1–3 : Introduction et Plan	3
1 Section 1 • Getting Ready (Pages 4–6)	4
1.1 Page 4 — Les Kits JetMax	4
1.2 Page 5 — Zone de Travail (Tapis)	4
1.3 Page 6 — Démarrage et Connexion	4
2 Section 2 • Prise en Main Rapide (Pages 7–9)	5
2.1 Page 7 — Application Mobile WonderAi	5
2.2 Page 8 — Interface Robot Control (APP)	5
2.3 Page 9 — Réglage des Seuils de Couleur — Espace Lab	6
3 Section 3 • Présentation Globale du Robot (Pages 10–12)	6
3.1 Page 10 — Architecture Mécanique et Physique	6
3.2 Page 11 — Actionneurs, Capteurs et Électronique	7
3.3 Page 12 — Architecture de Communication	7
4 Section 4 • Modélisation Cinématique (Pages 13–17)	7
4.1 Page 13 — Paramètres de Denavit-Hartenberg (DH)	7
4.2 Page 14 — Matrices de Transformation Homogène	8

4.3	Page 15 — Modèle Géométrique Direct (MGD)	8
4.4	Page 16 — Modèle Géométrique Inverse (MGI)	8
4.5	Page 17 — Singularités et Validation Numérique	8
5	Section 5 · Configuration Jetson Nano (Pages 18–19)	9
5.1	Page 18 — Système d'Exploitation et Pile IA	9
5.2	Page 19 — Validation Matérielle	9
6	Section 6 · Framework ROS (Pages 20–22)	9
6.1	Page 20 — Introduction à ROS sur JetMax	9
6.2	Page 21 — Concepts Fondamentaux ROS	9
6.3	Page 22 — Démonstrations et Outils de Débogage	10
7	Section 7 · Trajectoires et PID (Pages 23–25)	10
7.1	Page 23 — Planification de Trajectoires Cartésiennes	10
7.2	Page 24 — Régulation par PID Discret	10
7.3	Page 25 — Résultats et Validation des Trajectoires	11
8	Section 8 · Vision Artificielle (Pages 26–28)	11
8.1	Page 26 — Pipeline de Traitement d'Images (OpenCV)	11
8.2	Page 27 — Mapping Géométrique par Homographie	11
8.3	Page 28 — Séquence de Pick-and-Place	11
9	Section 9 · Contrôle PC et Programmation d'Actions (Pages 29–30)	12
9.1	Page 29 — Logiciel PC JetMax — Interface Navigateur	12
9.2	Page 30 — Programmation de Séquences d'Actions	12
10	Section 10 · Jeux de Vision IA (Pages 31–34)	12
10.1	Page 31 — Vue d'Ensemble des 9 Applications IA	13
10.2	Page 32 — Tri de Déchets — YOLOv5	13
10.3	Page 33 — Suivi de Visage — MTCNN + PID	13
10.4	Page 34 — Tri de Couleurs et Séquencement d'Objets	14
11	Section 11 · Outils Terminaux (Pages 35–36)	14
11.1	Page 35 — Porte-Stylet et Grande Pince	14
11.2	Page 36 — Ventouse Électromagnétique — Tri Vis et Écrous	14
12	Section 12 · Cours de Programmation Avancée (Pages 37–39)	15
12.1	Page 37 — Cours Avancés Jetson Nano	15
12.2	Page 38 — Carte d'Extension JetMax	15
12.3	Page 39 — Cours d'Extension Capteurs et Modules	15
13	Section 13 · Bases d'OpenCV et Vision (Pages 40–42)	16
13.1	Page 40 — Principes et Représentation des Images	16
13.2	Page 41 — Pipeline de Traitement d'Images en Couleur Lab	16
13.3	Page 42 — Contours et Suivi de Cible	16
14	Section 14 · Deep Learning Avancé et YOLOv5 (Pages 43–45)	17
14.1	Page 43 — Orthographe de Mots avec YOLOv5	17
14.2	Page 44 — Entraînement de Modèle YOLOv5 Personnalisé	17
14.3	Page 45 — Identification de Masques en Direct et Feedback	17
15	Section 15 · Conclusion — Page 46	18
15.1	Réalisations validées	18

15.2 Perspectives d'évolution	18
Annexe — Récapitulatif des pages par section	19

Pages 1–3 : Introduction et Plan

Page 1 — Page de titre

Page de titre en fond bleu foncé avec bandeau teal. Informations affichées : *Robot Hiwonder JetMax — Bras manipulateur articulé intelligent sous NVIDIA Jetson Nano et ROS*. Auteur : Projet de Robotique Appliquée, Arts et Métiers Campus Rabat, 2025–2026.

Page 2 — Introduction : Contexte et Objectifs

Contexte : Ce projet s'inscrit dans le module de Robotique Appliquée. L'objectif est de prendre en main le robot Hiwonder **JetMax** et de construire une chaîne complète allant de la modélisation mathématique jusqu'à une application de vision artificielle en temps réel.

Objectifs pédagogiques (5 points) :

1. Modéliser le robot (cinématique directe et inverse)
2. Configurer l'environnement embarqué (Jetson Nano)
3. Programmer sous ROS (nœuds, topics, services)
4. Générer et asservir des trajectoires (régulateur PID)
5. Implémenter une application de vision et de tri d'objets

Technologies utilisées :

Domaine	Technologie
Matériel	JetMax, servos HTS-35H, caméra HD 120°
Calcul	NVIDIA Jetson Nano (GPU Maxwell 128 cœurs)
Système	Ubuntu 18.04 LTS + JetPack SDK
Middleware	ROS Noetic / Melodic
Vision	OpenCV 4.5 (CUDA), HSV, Homographie
Langage	Python 3.6
Simulation	Matplotlib 3D, Gazebo

Résultat attendu : Un robot capable de **détecter** un objet par vision, **planifier** sa saisie et l'exécuter avec précision millimétrique.

Page 3 — Plan de la Présentation

Le plan est organisé en trois colonnes thématiques couvrant les 11 sections :

- **Mise en route** : Getting Ready · Présentation du Robot · Prise en Main Rapide · Contrôle PC & Actions
- **Fondements techniques** : Modélisation Cinématique · Configuration Jetson Nano · Framework ROS · Trajectoires & PID
- **Applications IA** : Vision Artificielle · Jeux de Vision IA · Outils Terminaux

1. Section 1 · Getting Ready (Pages 4–6)

1.1 Page 4 — Les Kits JetMax

Présentation des trois niveaux de kits commerciaux et du matériel embarqué.

Kit	Contenu principal	Usage ciblé
Starter	Bras + 4 servos HTS-35H	Cinématique de base
Standard	+ Ventouse à air + Caméra HD 120°	Vision IA
Advanced	+ Grande pince + Porte-stylo + Aimant	Applications complètes

Composants communs :

- **Jetson Nano B01** — GPU Maxwell 128 cœurs CUDA
- **Caméra HD grand angle** 120°, 1280×720 px, interface USB
- **Carte d'extension JetMax** — MCU STM32 dédié
- **Servomoteurs HTS-35H** — engrenages métalliques, couple 15 kg·cm

Cinq outils terminaux interchangeables :

1. **Ventouse à air** (défaut) — pression négative, objets plats
2. **Grande pince mécanique** — classification fruits/légumes (GoogLeNet)
3. **Ventouse électromagnétique** — pièces ferromagnétiques (vis, écrous)
4. **Porte-stylo** — dessin, écriture, expression d'émotions
5. **Petite pince** — objets fins

1.2 Page 5 — Zone de Travail (Tapis)

Le tapis officiel fourni avec le kit est divisé en quatre zones fonctionnelles utilisées par les algorithmes de vision et les séquences de tri.

Zone	Rôle
(1) Reconnaissance visuelle	Surface de dépôt des objets à analyser par la caméra
(2) Tri	Cases Rouge / Vert / Bleu + emplacements cartes déchets
(3) Placement robot	Destination de dépose avec coordonnées fixes calibrées
(4) Séquencement	Emplacement des cartes alphabet/chiffres pour l'IA

Note technique : Le tapis imprimé définit les repères physiques utilisés lors de la calibration par homographie. Les coordonnées de ses coins sont mesurées en millimètres et constituent les 4 points d'apprentissage de la matrice H .

1.3 Page 6 — Démarrage et Connexion

Procédure de démarrage (4 étapes) :

1. Connecter l'alimentation **DC 12 V / 5 A**

2. Basculer l'interrupteur en position **ON**
3. Attendre **3 bips sonores** $\Rightarrow$ démarrage réussi
4. Le Jetson Nano crée le hotspot Wi-Fi **HW-XXXXXXX** (mode AP)

Signification des 3 bips : Confirmation que le firmware MCU (STM32) est chargé et que le bus série vers les servos HTS-35H est actif et prêt à recevoir des commandes.

Modes de connexion réseau :

- **Mode AP** (par défaut) : hotspot autonome **HW-XXXXXXX**, IP fixe **192.168.149.1**
- **Mode Station (LAN)** : connexion à un réseau existant, IP dynamique DHCP

Outils d'accès distant :

- **SSH** : accès console sécurisé
- **NoMachine (NX)** : bureau graphique distant (RViz, débogage)
- **Navigateur** <http://192.168.149.1> : logiciel PC JetMax intégré

2. Section 2 · Prise en Main Rapide (Pages 7–9)

2.1 Page 7 — Application Mobile WonderAi

Installation : iOS (App Store) ou Android (APK officiel Hiwonder). Connexion directe au hotspot **HW-XXXXXXX** ou via réseau LAN.

Calibration avec AprilTag :

1. Placer le bloc **AprilTag** à position connue sur le tapis
2. L'application détecte automatiquement la balise et calcule la transformation pixel $\rightarrow$ repère physique
3. Valider : toutes les coordonnées sont désormais en millimètres dans le repère réel du bras

5 modes de l'application WonderAi :

#	Mode	Fonction
1	Position Calibration	Calibrage initial avec AprilTag
2	Robot Control	Pilotage manuel avec caméra live
3	Waste Sorting	Tri automatique de déchets (YOLOv5 IA)
4	Color Sorting	Tri par couleur rouge / vert / bleu
5	Block Stacking	Empilement automatique de blocs

2.2 Page 8 — Interface Robot Control (APP)

L'interface Robot Control est divisée en **5 zones** :

1. **Vue caméra live** — flux vidéo HD temps réel (Eye-in-Hand)

2. **Contrôle souris** — cliquer sur l'image cible une position ; le MGI calcule automatiquement les angles
3. **Contrôle cartésien** — boutons $\pm X$, $\pm Y$, $\pm Z$ pour déplacements pas à pas
4. **Sliders articulaires** — réglage individuel de chaque servo par son ID (1 à 4)
5. **Contrôle effecteur** — ventouse ON/OFF ou pince ouvrir/fermer

Point clé : Le mode souris permet à l'utilisateur de cliquer directement sur l'image caméra. La chaîne Vision $\rightarrow$ MGI $\rightarrow$ Servos est totalement transparente — c'est une interaction naturelle sans connaissance en robotique requise.

2.3 Page 9 — Réglage des Seuils de Couleur — Espace Lab

L'espace colorimétrique Lab :

Canal	Signification	Plage
L	Luminosité	0 (noir) $\rightarrow$ 255 (blanc)
a	Axe Vert $\leftrightarrow$ Rouge	0 $\rightarrow$ 255
b	Axe Bleu $\leftrightarrow$ Jaune	0 $\rightarrow$ 255

Avantage fondamental : Les canaux a et b (couleur pure) sont structurellement indépendants du canal L (luminosité). Contrairement à RGB où les 3 canaux varient ensemble avec l'éclairage, et à HSV où la saturation peut fausser sur les reflets, Lab sépare physiquement couleur et lumière.

Procédure de calibration Lab (4 étapes) :

1. Placer l'objet cible devant la caméra dans les conditions réelles
2. Augmenter **a_min** jusqu'à ce que le masque devienne **blanc uniforme** sur l'objet
3. Ajuster **L** et **b** selon l'éclairage ambiant
4. Vérifier la robustesse en déplaçant l'objet dans le champ

Seuils Lab typiques pour le rouge :

Canal	Minimum	Maximum
L (luminosité)	0	255
a (vert-rouge)	150	255
b (bleu-jaune)	0	255

3. Section 3 · Présentation Globale du Robot (Pages 10–12)

3.1 Page 10 — Architecture Mécanique et Physique

Structure mécanique : Robot manipulateur articulé en série, architecture anthropomorphe 3R + axe outil. **4 degrés de liberté actifs :**

1. θ_1 — Base : rotation azimutale (couverture 240°)

2. θ_2 — Épaule : extension verticale
3. θ_3 — Coude : inclinaison de l'avant-bras
4. θ_4 — Effecteur : orientation de l'outil

Matériau : alliage d'aluminium anodisé (légèreté + rigidité).

Spécifications opérationnelles :

Paramètre	Valeur
Portée maximale	289 mm
Couverture angulaire	240°
Charge utile	450 g
Répétabilité	±1 mm
Poids total	≈ 1,6 kg

3.2 Page 11 — Actionneurs, Capteurs et Électronique

Servomoteurs HTS-35H : engrenages métalliques (zéro jeu), couple 15 kg·cm @ 7,4 V, bus série half-duplex 115 200 bps en daisy-chain, rétroaction temps réel (position, température, tension).

Capteurs :

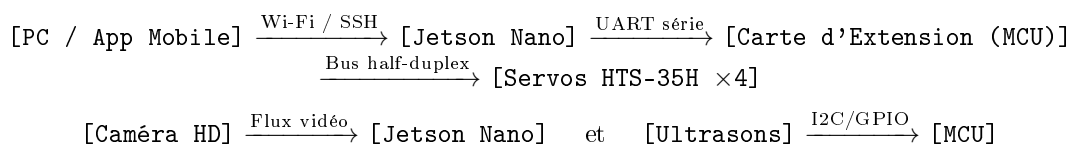
- Caméra HD 120° — Eye-in-Hand, 1280×720 px, USB
- Capteur ultrasonique lumineux — portée 2–400 cm, LED RGB (I2C/GPIO)

NVIDIA Jetson Nano B01 : CPU Quad-core ARM Cortex-A57 @ 1,43 GHz · GPU Maxwell 128 cœurs CUDA · RAM 4 GB LPDDR4 partagée · OS Ubuntu 18.04 + JetPack.

Alimentation & carte d'extension (STM32) : entrée DC 12 V / 5 A (60 W), tensions dérivées 7,4 V (servos), 5 V (Jetson), 3,3 V (capteurs).

3.3 Page 12 — Architecture de Communication

Diagramme TikZ reliant 6 composants :



Flux global : Vision $\rightarrow$ Planification MGI (Jetson) $\rightarrow$ Exécution (servos via MCU).

4. Section 4 · Modélisation Cinématique (Pages 13–17)

4.1 Page 13 — Paramètres de Denavit-Hartenberg (DH)

Liaisons physiques : $L_1 = 92,54$ mm (Base→Épaule), $L_2 = 128,40$ mm (Épaule→Coude), $L_3 = 144,80$ mm (Coude→Effecteur).

Table DH standard :

i	θ_i	d_i	a_i	α_i
1	θ_1	$L_1 = 92,54 \text{ mm}$	0	90
2	θ_2	0	$L_2 = 128,40 \text{ mm}$	0
3	θ_3	0	$L_3 = 144,80 \text{ mm}$	0

4.2 Page 14 — Matrices de Transformation Homogène

Matrice générale de passage du repère $i-1$ au repère i :

$$T_i^{i-1} = \begin{bmatrix} c_i & -s_i c_{\alpha_i} & s_i s_{\alpha_i} & a_i c_i \\ s_i & c_i c_{\alpha_i} & -c_i s_{\alpha_i} & a_i s_i \\ 0 & s_{\alpha_i} & c_{\alpha_i} & d_i \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Joint 1 ($\alpha_1 = 90, a_1 = 0$) : T_1^0 contient le terme L_1 sur la troisième ligne.

Joints 2 et 3 ($\alpha_i = 0$) : matrices de rotation plane avec $a_i c_i$ et $a_i s_i$ en translation.

4.3 Page 15 — Modèle Géométrique Direct (MGD)

Produit matriciel global : $T_3^0 = T_1^0 \cdot T_2^1 \cdot T_3^2$.

Équations de position de l'effecteur :

$$\begin{aligned} x &= \cos \theta_1 \cdot (L_2 \cos \theta_2 + L_3 \cos(\theta_2 + \theta_3)) \\ y &= \sin \theta_1 \cdot (L_2 \cos \theta_2 + L_3 \cos(\theta_2 + \theta_3)) \\ z &= L_1 + L_2 \sin \theta_2 + L_3 \sin(\theta_2 + \theta_3) \end{aligned}$$

Portée radiale : $r = L_2 \cos \theta_2 + L_3 \cos(\theta_2 + \theta_3)$, d'où $x = r \cos \theta_1$ et $y = r \sin \theta_1$.

4.4 Page 16 — Modèle Géométrique Inverse (MGI)

Étape 1 — Angle de la base : $\theta_1 = \text{atan2}(y, x)$

Étape 2 — Angle du coude (elbow-up) :

$$r = \sqrt{x^2 + y^2}, \quad z' = z - L_1, \quad D = \frac{r^2 + z'^2 - L_2^2 - L_3^2}{2L_2L_3}, \quad \theta_3 = \text{atan2}(\sqrt{1 - D^2}, D)$$

Étape 3 — Angle de l'épaule :

$$\theta_2 = \text{atan2}(z', r) - \text{atan2}(L_3 \sin \theta_3, L_2 + L_3 \cos \theta_3)$$

4.5 Page 17 — Singularités et Validation Numérique

Zones singulières :

- **Extension maximale** : $\theta_3 = 0$, portée maximale $r = 273,20 \text{ mm}$, perte d'un DDL
- **Singularité de base** : $x = y = 0$, θ_1 indéterminé

Validation Python (cinématique_sim.py) : Erreur de reconstruction MGD◦MGI : $\varepsilon \approx 10^{-15} \text{ mm}$ (seuil physique : $\pm 1 \text{ mm}$). Précision 10^{12} fois supérieure à la tolérance mécanique.

5. Section 5 · Configuration Jetson Nano (Pages 18–19)

5.1 Page 18 — Système d’Exploitation et Pile IA

- OS : **Ubuntu 18.04 LTS** + NVIDIA JetPack
- **CUDA Toolkit 10.2** — compilation et exécution GPU
- **cuDNN 8.2** — primitives Deep Learning optimisées
- **TensorRT** — inférence temps réel de réseaux de neurones
- **OpenCV 4.5.0** compilée avec support CUDA
- **Python 3.6** — langage principal d’implémentation
- Réseau : Mode AP (192.168.149.1), Mode Station, SSH, NoMachine

5.2 Page 19 — Validation Matérielle

- **UART** : port `/dev/ttyTHS1` @ 115 200 bps — envoi de trames de contrôle aux servos validé
- **Caméra USB** : `/dev/video0` — capture V4L2 via `cv2.VideoCapture(0)` validée
- **GPIO** : bibliothèque `Jetson.GPIO` — pilotage broche 12 (buzzer) validé

Bilan : tous les périphériques opérationnels ⇒ **système prêt pour ROS et Vision IA.**

6. Section 6 · Framework ROS (Pages 20–22)

6.1 Page 20 — Introduction à ROS sur JetMax

ROS est un middleware standardisé fournissant gestion du matériel, passage de messages inter-processus et gestion de packages. Version ciblée : ROS Noetic / Ubuntu 20.04.

Workspace Catkin :

```
mkdir -p ~/catkin_ws/src
cd ~/catkin_ws && catkin_make
source ~/catkin_ws/devel/setup.bash
```

3 packages officiels JetMax :

1. `jetmax_description` — URDF : géométrie, masses, inerties
2. `jetmax_control` — interface série + services cinématiques
3. `jetmax_gazebo` — simulation physique Gazebo

6.2 Page 21 — Concepts Fondamentaux ROS

Master et Nœuds : `roscore` est le registre central de nommage (doit démarrer en premier). Les nœuds sont des processus indépendants (Python ou C++).

Topics (asynchrone Pub/Sub) : Canal unidirectionnel, type `sensor_msgs/JointState`, fréquence cible 10 Hz.

Services (synchrone Req/Res) : Mécanisme bloquant bidirectionnel. Services cinématiques JetMax :

- `/forward_kinematics` : $(x, y, z) \rightarrow (\theta_1, \theta_2, \theta_3)$
- `/inverse_kinematics` : $(\theta_1, \theta_2, \theta_3) \rightarrow (x, y, z)$

6.3 Page 22 — Démonstrations et Outils de Débogage

Nœuds développés :

- `joint_publisher.py` — consignes sinusoïdales à 10 Hz
- `joint_subscriber.py` — affichage des angles reçus en degrés
- `kinematics_service.py` — serveur MGD/MGI déporté

Outils CLI de débogage : `rostopic list/echo/hz`, `rosservice list/call`, `rqt_graph` (topologie du réseau de nœuds).

7. Section 7 · Trajectoires et PID (Pages 23–25)

7.1 Page 23 — Planification de Trajectoires Cartésiennes

Trajectoire rectiligne : $P(t) = A + s(t) \cdot (B - A)$, $s(t) \in [0, 1]$.

Trajectoire circulaire : $x(t) = x_C + R \cos(\omega t)$, $y(t) = y_C + R \sin(\omega t)$, $z(t) = z_0$ (hauteur constante).

Choix retenu : espace cartésien. Le chemin est discrétisé et le MGI est appliqué point par point, garantissant que l'outil suit rigoureusement la trajectoire physique souhaitée (contrairement à l'espace articulaire qui produit un chemin courbe imprévisible).

7.2 Page 24 — Régulation par PID Discret

Chaque articulation possède son propre régulateur :

$$u(k) = K_p e(k) + K_i \sum_j e(j) \Delta t + K_d \frac{e(k) - e(k-1)}{\Delta t}$$

Terme	Param.	Rôle
Proportionnel	$K_p = 8,0$	Ralliement rapide (oscillations si trop élevé)
Intégral	$K_i = 0,5$	Annule l'erreur statique, compense la gravité
Dérivé	$K_d = 0,1$	Amortit les oscillations, limite le dépassement

Paramètres de simulation : $\Delta t = 0,025$ s (40 Hz), durée 5 s, constante de temps moteur $\tau = 0,08$ s. Mécanismes de sécurité : anti-windup + saturation de vitesse.

7.3 Page 25 — Résultats et Validation des Trajectoires

Métrique	Valeur
Correction d'une perturbation de 41,3 mm	< 1,2 s
Erreur régime établi (cercle)	0,015 mm
Erreur régime établi (ligne)	0,018 mm
Erreur finale ($t = 5$ s)	0,009 mm

Un graphe pgfplots illustre la décroissance exponentielle de l'erreur avec la tolérance physique de 1 mm tracée en pointillés. Le système est 50 fois plus précis que la tolérance physique du robot.

8. Section 8 · Vision Artificielle (Pages 26–28)

8.1 Page 26 — Pipeline de Traitement d'Images (OpenCV)

Espace HSV : Conversion BGR $\rightarrow$ HSV. Le masque binaire $M(u, v)$ est défini par :

$$M(u, v) = \begin{cases} 1 & \text{si } H \in [H_{\min}, H_{\max}] \wedge S \in [S_{\min}, S_{\max}] \wedge V \in [V_{\min}, V_{\max}] \\ 0 & \text{sinon} \end{cases}$$

Avantage : Le canal Hue est robuste aux variations d'éclairage.

Extraction du centroïde (3 étapes) :

1. Filtrage morphologique (érosion + dilatation)
2. Détection de contours OpenCV
3. Moments d'image : $u_c = m_{10}/m_{00}$, $v_c = m_{01}/m_{00}$

8.2 Page 27 — Mapping Géométrique par Homographie

La table est un plan horizontal z_{table} fixe $\Rightarrow$ **homographie plane** H (3×3) :

$$\begin{bmatrix} x_w \\ y_w \\ w \end{bmatrix} = H \cdot \begin{bmatrix} u_c \\ v_c \\ 1 \end{bmatrix} \quad \Rightarrow \quad x_r = \frac{x_w}{w}, \quad y_r = \frac{y_w}{w}$$

Calibration : 4 points de repère (pixel $\leftrightarrow$ physique) $\rightarrow$ `cv2.findHomography`.

Exemple validé : centroïde (260, 205) px $\rightarrow$ $X_r = 177,30$ mm, $Y_r = 35,00$ mm.

Pipeline complet : Pixels $(u_c, v_c) \rightarrow$ Homographie $H \rightarrow$ Robot $(x_r, y_r) \rightarrow$ MGI $\rightarrow$ Angles servos.

8.3 Page 28 — Séquence de Pick-and-Place

Séquence en 6 phases pour éviter les collisions latérales :

Phase	Position	z	Effecteur
1. Home	(120, 0, 150) mm	150 mm	—
2. Approche	$(x_r, y_r, 70)$ mm	70 mm	—
3. Saisie	$(x_r, y_r, 20)$ mm	20 mm	Pompe ON
4. Levage	$(x_r, y_r, 70)$ mm	70 mm	Pompe ON
5. Dépose	(120, −120, 80) mm	80 mm	Pompe OFF
6. Retour (Home)	(120, 0, 150) mm	150 mm	—

Résultats de `vision_sim.py` : détection validée, mapping opérationnel, MGI résolu pour toutes les poses, visualisation 3D Matplotlib confirmée.

9. Section 9 · Contrôle PC et Programmation d'Actions (Pages 29–30)

9.1 Page 29 — Logiciel PC JetMax — Interface Navigateur

Accès via navigateur web à <http://192.168.149.1>, aucune installation requise sur le PC.

Mode Contrôle à Distance : identique à l'APP mobile (vue caméra, contrôle souris, boutons cartésiens, sliders par ID servo, effecteur).

Mode Éditeur de Groupes d'Actions (8 étapes) :

1. Créer un nouveau groupe et lui donner un nom
2. Cliquer Reset → position initiale
3. Cliquer Add → ajouter une pose
4. Ajuster les sliders (mode articulaire ou cartésien)
5. Enregistrer la pose dans la séquence
6. Répéter 3–5 pour chaque étape
7. Sauvegarder le groupe d'actions complet

9.2 Page 30 — Programmation de Séquences d'Actions

Deux modes de réglage des poses :

- **Mode servo individuel** : contrôle angle par angle (ID 1–4), précision maximale pour les gestes fins
- **Mode coordonnées cartésiennes** : entrer (X, Y, Z) en mm, MGI automatique transparent

Durée de transition : chaque pose possède une durée en millisecondes contrôlant la vitesse de déplacement entre poses. Plus grande = plus lent et plus fluide.

10. Section 10 · Jeux de Vision IA (Pages 31–34)

10.1 Page 31 — Vue d'Ensemble des 9 Applications IA

Applications visuelles (YOLOv5 / Lab + OpenCV) :

1. Tri de déchets — YOLOv5 multiclasse
2. Séquencement d'objets — YOLOv5 cartes lettres/chiffres
3. Tri de couleurs — seuillage Lab + `cv2.findContours`
4. Suivi d'objet — PID de centroïde en temps réel
5. Calcul numérique — reconnaissance de chiffres

Applications Deep Learning avancées :

6. Empilement de blocs — planification séquentielle par vision
7. Suivi de visage — MTCNN 3 étages + PID
8. Reconnaissance d'émotions — CNN facial
9. Reconnaissance de gestes — keypoints des mains

Architecture commune : Caméra HD → Modèle IA (YOLOv5/CNN) → Traitement OpenCV → Mapping pixel→Robot → MGI→Angles → Pick & Place.

10.2 Page 32 — Tri de Déchets — YOLOv5

Pipeline (5 étapes) :

1. Capture flux caméra HD
2. Inférence YOLOv5 : détecte et classifie (recyclable / biologique / autres)
3. Sélection de l'objet à la **plus haute confiance**
4. Homographie : pixel → coordonnées physiques sur la table
5. Pick-and-Place vers la case correspondante

Commandes ROS : `/waste_classification/enter` et `/waste_classification/set_running`.

Avantage YOLOv5 : détection + classification en une seule passe (one-shot), compatible avec le GPU Maxwell du Jetson Nano en temps réel.

Variante — Séquencement d'objets : YOLOv5 sur cartes alphabet/chiffres, sélection du plus petit objet en premier (min taille boîte englobante). Script : `alphabetically_main.py`.

10.3 Page 33 — Suivi de Visage — MTCNN + PID

Réseau MTCNN (3 étages en cascade) :

1. **P-Net** (Proposal) — proposition rapide de régions candidates
2. **R-Net** (Refine) — filtrage et affinement des candidats
3. **O-Net** (Output) — boîte finale + 5 points clés faciaux (yeux, nez, bouche)

Asservissement PID : Erreur $e_x = u_f - u_{\text{centre}}$, $e_y = v_f - v_{\text{centre}}$, correction angulaire $\Delta\theta_1$ (panoramique) et $\Delta\theta_2$ (tilt).

Prérequis : caméra en position **horizontale**. Commandes ROS : `/object_tracking/enter` et `/object_tracking/set_running_face "data: true"`.

10.4 Page 34 — Tri de Couleurs et Séquencement d'Objets

Tri de couleurs (Lab + OpenCV) :

1. BGR $\rightarrow$ Lab, application des seuils calibrés
2. `cv2.findContours` : extraction des formes
3. Plus grand contour = objet cible
4. Dépôt dans la case R/V/B selon la couleur détectée

Couleur	Canal Lab discriminant	Case de dépôt
Rouge	a élevé (> 150)	Case rouge
Vert	a faible (< 100)	Case verte
Bleu	b faible (< 100)	Case bleue

Commandes ROS : `/color_sorting/enter`, `/color_sorting/set_target "color_name: 'red'"`, `/color_sorting/set_running "data: true"`.

11. Section 11 · Outils Terminaux (Pages 35–36)

11.1 Page 35 — Porte-Stylo et Grande Pince

Porte-Stylo : Montage : 4 vis M3×8 + 2 vis M3×15. Applications : tracé géométrique (cercle, ligne), dessin d'émotions (IA), écriture de texte. La précision PID de 0,02 mm rend le tracé exploitable pour une écriture réelle.

Grande Pince — Classification GoogLeNet : Modèle PyTorch + GoogLeNet (Inception V1), 10 classes : pomme, banane, poivron, carotte, citron, laitue, mangue, pomme de terre, piment rouge, rien.

- `move_1()` : fruit $\rightarrow$ panier fruits
- `move_2()` : légume $\rightarrow$ panier légumes
- Script : `obj_classification.py`

11.2 Page 36 — Ventouse Électromagnétique — Tri Vis et Écrous

Principe : induction magnétique (pièces ferromagnétiques uniquement), activation/désactivation par GPIO.

Pipeline YOLOv5 (5 étapes) :

1. Inférence YOLOv5 : 2 classes (vis, écrou)
2. Calcul centroïde $\rightarrow$ coordonnées physiques
3. Déplacement vers la pièce

4. Activation aimant → saisie magnétique
5. Transport et dépôt selon le type

Positions de dépôt calibrées :

Type	x	y	z
Vis	120 mm	0 mm	140 mm
Écrou	−120 mm	0 mm	140 mm

Script : `screw_nut.py`. ROS : `/em_suction/enter` et `/em_suction/set_running`.

12. Section 12 · Cours de Programmation Avancée (Pages 37–39)

12.1 Page 37 — Cours Avancés Jetson Nano

Ce module couvre l'administration, le paramétrage réseau et la maintenance de l'unité de calcul embarquée NVIDIA Jetson Nano.

- **Configuration Système** : Gravure de l'image officielle d'Ubuntu 18.04 LTS intégrant le SDK NVIDIA JetPack. Configuration du Point d'Accès Wi-Fi local (Mode AP autonome) et mode Station.
- **Prise de Contrôle Distant** : Configuration de NoMachine (NX) pour déporter l'affichage graphique sous forme de bureau distant à haute cadence, indispensable pour afficher les outils visuels de ROS (RViz) ou OpenCV.
- **Transferts SFTP** : Utilisation du protocole SFTP pour le chargement des scripts Python de contrôle.
- **Sauvegarde & Restauration** (Leçon 10) : Clonage de la carte microSD intégrale et compression via `dd` et `gzip` pour restaurer instantanément le système après une panne ou une coupure de courant.

12.2 Page 38 — Carte d'Extension JetMax

La carte d'extension JetMax intègre un microcontrôleur STM32 dédié aux tâches de contrôle bas niveau en temps réel.

- **Servomoteurs HTS-35H** : Liaison série half-duplex en cascade à 115 200 bps. Chaque servo renvoie des informations critiques de rétroaction : position angulaire, température et tension de fonctionnement.
- **Actionneurs additionnels** : Gestion de plusieurs servomoteurs PWM standard pour les outils optionnels, contrôle de la puissance de moteurs DC et gestion du buzzer piezo intégré.

12.3 Page 39 — Cours d'Extension Capteurs et Modules

Ce module présente l'intégration pratique de capteurs interactifs dans des projets mécatroniques complets :

1. **Ventilateur intelligent** : Le bras détecte le visage par MTCNN, s'oriente face à lui par asservissement de position et active via GPIO un ventilateur monté sur l'effecteur.

2. **Scanner à ultrasons** : Balayage angulaire et lecture I2C de distance pour dresser une carte d'obstacles à l'écran.
3. **Matrice LED TM1640** : Classification CNN des expressions faciales et tracé de motifs dynamiques (smiley en cas de joie, croix sinon) sur matrice LED 8x16 via le contrôleur série TM1640.
4. **Reconnaissance de gestes** : Inférence des points clés de la main et affichage du compteur de doigts sur un afficheur à tubes numériques géré par GPIO.
5. **Contrôle tactile capacitif** : Interruption matérielle GPIO pour démarrer/réinitialiser les cycles industriels en toute sécurité.

13. Section 13 · Bases d'OpenCV et Vision (Pages 40–42)

13.1 Page 40 — Principes et Représentation des Images

OpenCV stocke les images sous forme de tableaux Numpy de dimension 2 (niveaux de gris 8 bits, $0 \rightarrow 255$) ou 3 (couleur BGR 24 bits, $H \times W \times 3$).

- **Ordre BGR** : OpenCV stocke par défaut les composantes de couleur dans l'ordre Bleu-Vert-Rouge (BGR) au lieu du RGB standard. Accès pixel : `image[y, x, c]`.
- **Slicing (ROI)** : Le découpage Numpy permet d'isoler une région d'intérêt (ROI) pour restreindre et accélérer les calculs.
- **Espaces Colorimétriques** : Comparatif des espaces BGR (sensible à la lumière), HSV (teinte isolée mais saturation variable sur les bords) et Lab (séparation physique de la luminance L et des axes de couleur pure a, b). L'espace colorimétrique Lab est idéal pour la robustesse face aux variations d'éclairage.

13.2 Page 41 — Pipeline de Traitement d'Images en Couleur Lab

La segmentation d'un objet coloré suit un pipeline en 4 étapes sous OpenCV :

1. **Flou Gaussien** : Réduction du bruit haute fréquence par convolution matricielle : `cv2.GaussianBlur(img, (5, 5), 5)`
2. **Conversion Lab** : Conversion d'espace via `cv2.cvtColor` avec le drapeau `COLOR_BGR2LAB`.
3. **Binarisation** : Seuillage par plage via `cv2.inRange` pour obtenir un masque noir et blanc.
4. **Lissage Morphologique** : Élimination des bruits d'aires (érosion `cv2.erode`) et bouchage des trous (dilatation `cv2.dilate`) à l'aide d'un élément rectangulaire structurant 3×3 (`cv2.MORPH_RECT`).

13.3 Page 42 — Contours et Suivi de Cible

- **Extraction** : Localisation des contours externes via la fonction `cv2.findContours`.
- **Filtrage d'Aire** : Calcul de l'aire géométrique par `cv2.contourArea` et rejet des surfaces de bruit inférieures à **1200 pixels**. Sélection du plus grand contour pour le suivi.

- **Feedback Graphique** : Marquage par cercles (`cv2.circle`), réticule central en forme de croix (`cv2.line`) et incrustation de texte dynamique (`cv2.putText`) sur le flux vidéo.

14. Section 14 · Deep Learning Avancé et YOLOv5 (Pages 43–45)

14.1 Page 43 — Orthographe de Mots avec YOLOv5

Le nœud ROS `alphabetically_main.py` implémente un cycle intelligent d'épellation physique de mots :

- **Inférence YOLOv5 TRT** : Inférence en temps réel sur les cartes d'alphabet disposées sur la table.
- **Logique Séquentielle** : Consigne du mot cible défini par `WORD_WANT` (ex : "DBRH"). La fonction Python `max()` sélectionne la carte de plus haute confiance détectée pour servir de référence. La fonction `min()` identifie la carte correspondante la plus proche de l'outil en calculant la distance euclidienne.
- **Cinématique Inverse (MGI)** : Conversion homographique $2D \rightarrow$ Coordonnées X, Y du robot $\rightarrow$ Calcul des angles articulaires requis $\rightarrow$ Succion/dépose en séquence pour épeler le mot.

14.2 Page 44 — Entraînement de Modèle YOLOv5 Personnalisé

Ce module décrit le pipeline de développement d'un modèle de détection de masques :

- **Dataset facemask** : Captures d'images au format 640×480 pixels stockées sous `JPEGImages`. Annotation manuelle en mode NX distant via l'outil graphique `labelImg` (classes `mask` et `nomask`) enregistrée en XML sous `Annotations`.
- **Pré-processing** : Partitionnement train/val/test (80/15/5) via `txt_gen.py` et conversion des annotations XML en format d'étiquetage YOLO normé `.txt` via `xml2yolo.py`. Création de la configuration `facemask.yaml`.
- **Optimisation GPU** : Entraînement PyTorch (`train.py`), conversion vers le format universel ONNX (`export.py`), puis compilation TensorRT (`onnx2trt`) pour obtenir une exécution temps réel fluide à FPS élevé sur le GPU Maxwell embarqué du robot.

14.3 Page 45 — Identification de Masques en Direct et Feedback

Le nœud ROS `face_mask_main.py` synchronise l'inférence IA avec des réactions physiques :

- **Abonnement Caméra** : Souscription au topic caméra `/usb_cam/image_rect_color`. Inférence en direct via le réseau optimisé TensorRT `best.trt` et affichage des boîtes de délimitation de couleur sur le bureau NoMachine.
- **Feedback Matrice LED TM1640** : Transmission de trames série GPIO vers le contrôleur TM1640. Si le visage détecté porte un masque $\rightarrow$ activation de la fonction `draw_smail()` pour dessiner un smiley souriant. Si le visage est non masqué $\rightarrow$ activation de `draw_cross()` affichant une croix rouge clignotante de signallement.

15. Section 15 · Conclusion — Page 46

15.1 Réalisations validées

#	Réalisation	Résultat quantitatif
1	Modélisation DH, MGD, MGI	Erreur reconstruction $\varepsilon < 10^{-14}$ mm
2	Matériel Jetson + servos + caméra	Tous les périphériques opérationnels
3	ROS Publisher, Subscriber, Service	Nœuds fonctionnels à 10 Hz
4	Trajectoires PID	Erreur régime établi $< 0,02$ mm
5	Vision HSV + Homographie + Pick-and-Place	Cycle complet validé en simulation

15.2 Perspectives d'évolution

- **Déploiement embarqué** sur le vrai robot JetMax physique
- Détecteur **YOLOv8** multi-classe pour de meilleures performances
- **Calibration intrinsèque** complète (pinhole + distorsion radiale)
- Planification sans collision avec **MoveIt !**
- **Apprentissage par renforcement (RL)** pour l'adaptation autonome

Annexe — Récapitulatif des pages par section

Pages	Section	Slides
1	Page de titre	1
2–3	Introduction + Plan	2
4–6	Getting Ready	3
7–9	Prise en Main Rapide (APP + Lab)	3
10–12	Présentation Globale du Robot	3
13–17	Modélisation Cinématique	5
18–19	Configuration Jetson Nano	2
20–22	Framework ROS	3
23–25	Trajectoires & PID	3
26–28	Vision Artificielle	3
29–30	Contrôle PC & Programmation d'Actions	2
31–34	Jeux de Vision IA	4
35–36	Outils Terminaux	2
37–39	Cours de Programmation Avancée	3
40–42	Bases d'OpenCV et Vision	3
43–45	Deep Learning Avancé et YOLOv5	3
46	Conclusion	1
Total		46 pages